

Chikitsa Finder

Smart Doctor Appointment & Booking System

Powered by Node.js, Express, and MongoDB



The Challenge vs. The Solution

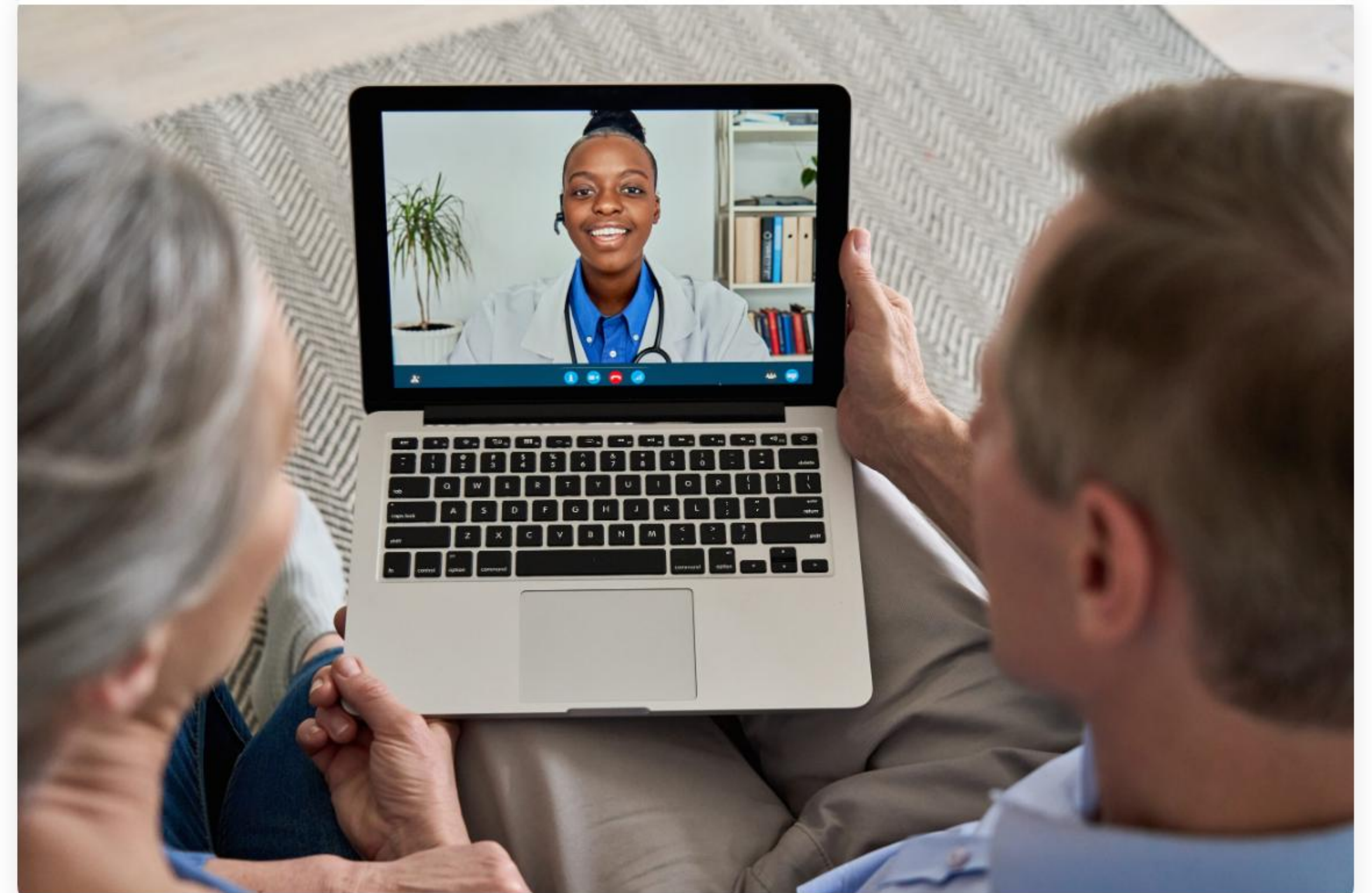
The Challenge

Finding the right specialist and booking an appointment often involves:

- Endless phone calls to clinics.
- Uncertain availability.
- Lack of immediate confirmation.

The Solution

Chikitsa Finder streamlines this process by providing a centralized platform to search, view doctor profiles, and book slots instantly with automated email verification.



Project Overview

A full-stack web application designed to bridge the gap between patients and healthcare providers through a seamless, digital interface.

Key Features



Smart Search

Filter doctors by name, city, or specialization instantly using the frontend search bar.



Instant Booking

Simple modal-based booking form that captures user details and appointment preferences.



Email Alerts

Automated confirmation emails sent via Nodemailer to ensure patients have a record of their booking.

Technology Stack



Frontend

HTML5, Bootstrap 5 for responsive design, and vanilla JavaScript for DOM manipulation and API calls.



Backend

Node.js environment with Express.js framework to handle API routing and middleware.



Database

MongoDB (NoSQL) for storing Doctor and Appointment records, managed via Mongoose.

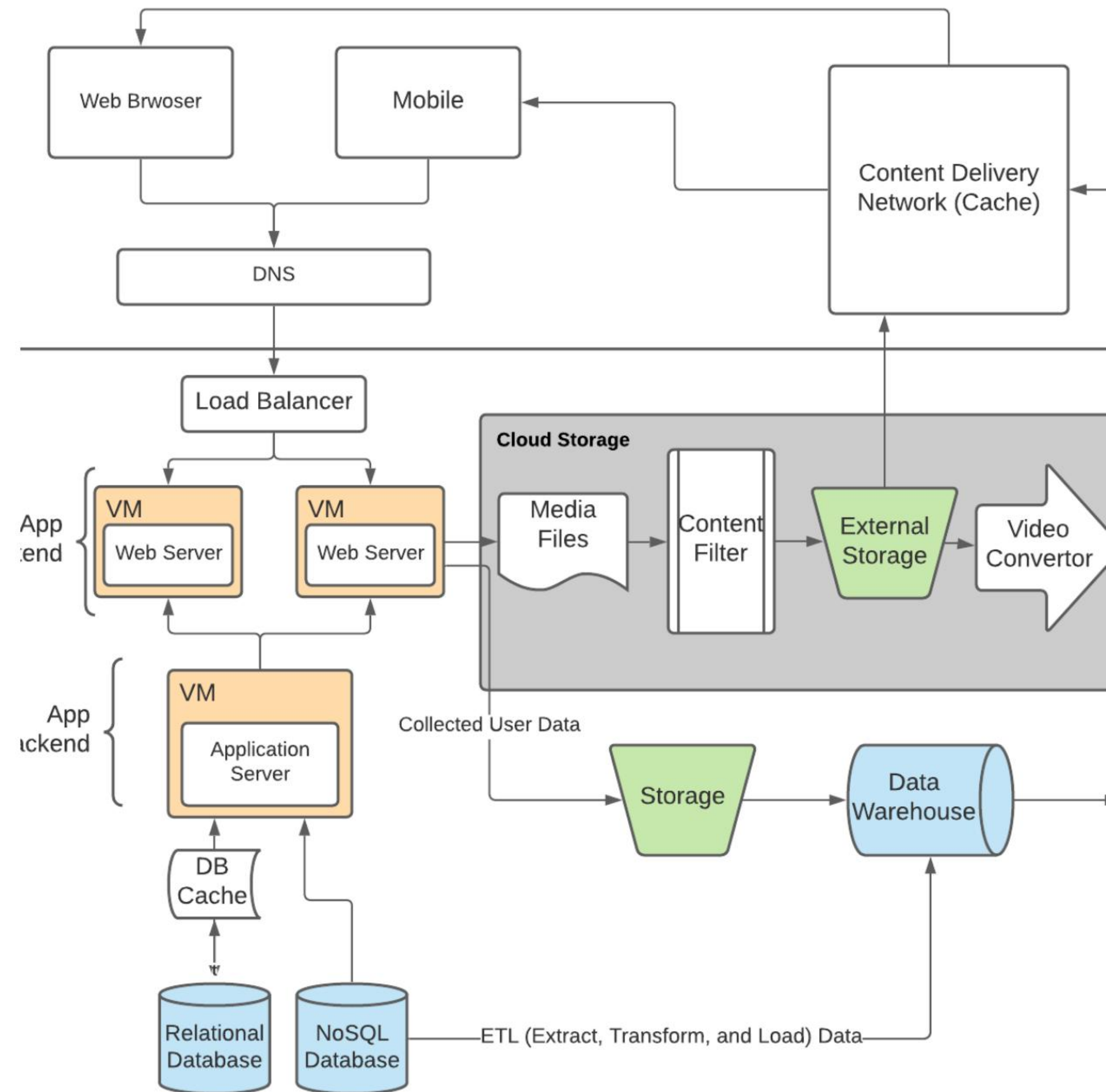
System Architecture

Client-Server Model

The application operates on two distinct ports to separate concerns:

- **Port 5501 (Frontend):** Serves the static UI files (HTML/CSS) to the user's browser.
- **Port 8000 (Backend):** Listen for API requests, connects to MongoDB, and processes emails.

Communication happens via asynchronous fetch() requests from the client to the server.



Backend Logic: Email Integration

Nodemailer Implementation

A critical feature is the integration of SMTP services. Using Google's "App Password" feature, the backend securely authenticates with Gmail to send notifications.

Environment variables (.env) are used to protect sensitive credentials like MAIL_USER and MAIL_PASS.

server.js

```
const transporter = nodemailer.createTransport({
  service: "gmail",
  auth: {
    user: process.env.MAIL_USER,
    pass: process.env.MAIL_PASS
  }
});

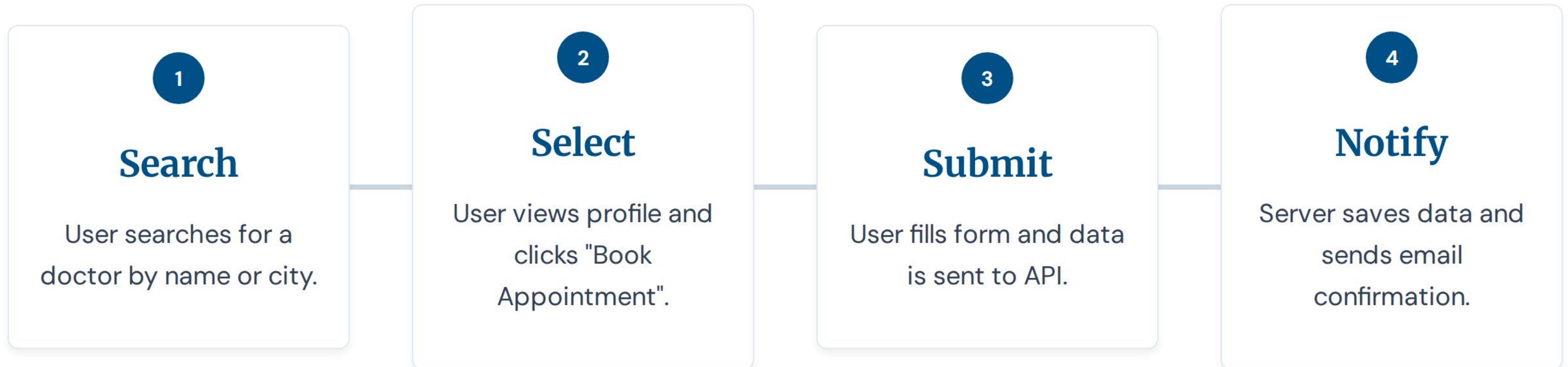
// Sending the email
await transporter.sendMail({
  from: "Chikitsa Finder",
  to: userEmail,
  subject: "Appointment Confirmed",
  html: `Your booking with ${doctorName} is confirmed.`
});
```


Database Schema (Mongoose)

Model	Fields	Purpose
Doctor	Name, Specialization, Location, Image, Description	Stores static data about medical professionals to populate the search list.
Appointment	Name, Email, Phone, Doctor Name, Date, Message	Captures transactional data from the booking form for records and notifications.

Data is stored in a structured JSON format within MongoDB collections.

User Workflow



Conclusion & Future Scope

Current Status

Chikitsa Finder is a fully functional MVP that successfully handles the core loop of finding and booking doctors.

Future Roadmap

- **Admin Dashboard:** For doctors to manage their own schedules.
- **Payment Gateway:** Integration of Razorpay/Stripe for consultation fees.
- **SMS Notifications:** Using Twilio for instant mobile



Image Sources



<https://tiasante.com/wp-content/uploads/2021/07/iStock-1297569926-scaled.jpg>

Source: tiasante.com



https://miro.medium.com/1*MbtHljk_Y-HT_KwwYmaQVQ.png

Source: [medium.com](https://miro.medium.com)



https://media.istockphoto.com/id/531475568/photo/patient-and-medical-staff.jpg?s=612x612&w=0&k=20&c=KqciFJ4FCJMwY04UcJx1ZJe_316jM65GAUImbTmGzBk=

Source: www.istockphoto.com



https://static.vecteezy.com/system/resources/previews/008/551/639/non_2x/abstract-health-medical-science-healthcare-icon-and-world-digital-technology-doctor-concept-modern-innovation-treatment-medicine-on-hi-tech-future-blue-background-vector.jpg

Source: www.vecteezy.com